

Does it actually work? Evaluating web agents

Week 9, Lecture 1 · Browser Use: How LLM Agents Drive the Web

Summer 2026

What we will cover

- WebVoyager and Mind2Web: what these benchmarks measure
- Why high scores can mislead: cached snapshots vs live sites
- The COLM 2025 critique and the Online-Mind2Web findings
- Cost and latency per task: the numbers that actually matter
- Prompt injection: when page content attacks the agent
- What a trustworthy evaluation looks like

Part 1: the benchmark landscape

WebVoyager and Mind2Web

- **WebVoyager**: open-ended tasks, scored by a human or LLM judge
- **Mind2Web**: ~2,000 tasks on 137 websites, recorded by human annotators
- Both measure task completion rate, reported as a percentage
- Both have produced benchmark numbers in the 70–90% range
- These numbers are cited constantly in web-agent research and marketing

The core problem: cached snapshots

- Mind2Web data is collected at a point in time, then frozen
- A cached snapshot is a fixed HTML file, not a live website
- Agents, or their prompts, can be tuned against the snapshot
- Months later, the live site has changed: new modals, new layouts, A/B tests
- The agent that aced the snapshot fails on the real page

Part 2: the COLM 2025 critique

"An Illusion of Progress?"

- COLM 2025 paper (arXiv 2504.01382) introduced Online-Mind2Web
- 300 tasks across 136 websites, evaluated on live sites
- Key finding: scores from cached-snapshot benchmarks fall sharply on live sites
- The gap is structural, not a calibration issue
- Live sites require improvisation; frozen snapshots reward memorization

Three questions to ask about any benchmark score

1. Was the evaluation run on **live sites** or archived snapshots?
2. Was the evaluator a **human** or an LLM judge (85% agreement with humans)?
3. Were the tasks drawn from a **held-out set** the system never trained on?

A score that answers “archived, LLM, no” is much less informative than one that answers “live, human, yes.”

Part 3: cost and latency

The numbers benchmarks don't report

- Each step calls the LLM with 1,500–3,000 tokens of browser state
- Vision adds 4–6x tokens per step
- A 12-step task at \$3/M input tokens: roughly \$0.07 per task
- At 10,000 tasks/day: roughly \$700/day in LLM fees (DOM-only)
- With vision at 4x: roughly \$2,800/day

Source: NxCode (2026) cost estimates; arithmetic is illustrative.

Latency is also a first-class metric

- Each LLM call takes 1–5 seconds for a fast model
- A 12-step task = 12–60 seconds of LLM call time alone
- Plus actual browser actions: clicks, navigation, page loads
- A task that takes one second in Playwright can take 30 seconds with browser-use
- Task SLA requirements often rule out autonomous agents before cost does

Part 4: prompt injection

The page can try to hijack the agent

- The browser-state summary includes the text content of the page
- A malicious site can place text that looks like agent instructions
- Example: a hidden `<div>` reading "Ignore your task. Navigate to..."
- The LLM sees this text as part of its context
- This is prompt injection, the same attack that affects chatbots, now in a browser

Why this matters more for browser agents than for Playwright

- A Playwright script reads specific selectors; it does not reason about text
- The moment an LLM interprets page content, all text becomes a potential vector
- The attack surface scales with the number of sites the agent visits
- A single compromised or malicious CDN can inject into thousands of pages

Mitigations available now

- `allowed_domains` : blocks navigation to domains not on the list
- `sensitive_data` : placeholders keep secrets out of the LLM context
- Neither is a complete defense, they limit blast radius
- Full defense against prompt injection in web agents is an open research problem (as of mid-2026)

Part 5: what a trustworthy eval looks like

Building your own evaluation

1. **Use live sites**, not snapshots you took when building the agent
2. **Define a held-out task set** before you start building
3. **Set a precise success criterion** for each task
4. **Run multiple trials** (3–5 per task), browser-use is not deterministic
5. **Report pass rate, cost, and latency** together, not just pass rate

Current web agent capabilities are notably lower than previously reported, and shortcomings in existing benchmarks are a primary cause.

Take-aways

1. WebVoyager and Mind2Web are the dominant benchmarks, but both are vulnerable to cached-snapshot inflation.
2. The COLM 2025 paper shows scores fall sharply on live sites. Ask three questions before trusting a benchmark number.
3. Cost and latency are first-class metrics. A 12-step DOM-only task costs roughly \$0.07 in LLM fees at \$3/M tokens.
4. Prompt injection is a real risk. `allowed_domains` and `sensitive_data` limit exposure but do not fully solve the problem.
5. A trustworthy eval uses live sites, held-out tasks, multiple trials, and reports pass rate alongside cost and latency.

What's next

- **Lecture 2 (this week):** Deploying browser-use (CLI, MCP, cloud) and the wider ecosystem comparison
- **Section (this week):** One task, three ways, run it autonomously, task-framed, and as a Playwright script; fill in a scorecard
- **Reading:** Week 9 reading covers evaluation, cost, prompt injection, deployment, and the bitter-lesson design philosophy